

Vigilância de Partições Retangulares

Relatório do Trabalho Prático

Unidade Curricular	Métodos de Apoio à Decisão
Participantes	Afonso Araújo - up202305181 Bruno Barros – up202305457 Francisco Moreira - up202306691
Ferramentas	Python 3 · OR-Tools (CP-SAT/SCIP) · SWI-Prolog CLP(FD)

Resumo do relatório

Este relatório descreve a resolução do problema de **Vigilância de Partições Retangulares**: colocar o menor número possível de guardas nos vértices de uma partição de modo a que cada retângulo de um subconjunto $\Pi' \subseteq \Pi$ seja vigiado. São avaliadas quatro famílias de métodos — **greedy**, **programação inteira e por restrições** (MAC/AC-3 manual, OR-Tools ILP/CP-SAT e SWI-Prolog CLP(FD)), **programação dinâmica** e **extensões** (coloração de guardas e alcance alargado D).

1. Descrição do Problema

Dada uma partição Π de um rectângulo em sub-retângulos, um guarda colocado no vértice v vigia todos os retângulos que têm v como canto. O objectivo é encontrar um conjunto de vértices de cardinalidade mínima que garanta a cobertura de $\Pi' \subseteq \Pi$. O problema admite **cobertura total** ($\Pi' = \Pi$) e **cobertura parcial** ($\Pi' \subset \Pi$). O grafo de incidência bipartido $G = (V \cup R, E)$ representa o problema; **vertex_to_rects**[v] e **rect_to_verts**[r] são as estruturas centrais, comuns a todos os módulos.

1.1 Instâncias de Teste

Ficheiro	Inst.	Rects	Vértices	Tipo de Partição
inst_grid.txt	6	4–20	9–30	Grelhas regulares $n \times m$
inst_small.txt	10	6–10	12–20	Irregulares pequenas
inst_1d.txt	10	2–9	4–11	Partições lineares (1D)
inst_medium.txt	10	10–24	18–35	Irregulares médias
inst_large.txt	5	28–63	42–90	Irregulares grandes
inst_adversarial.txt	8	6–20	14–30	Instâncias adversariais
inst_original.txt	4	7	11–13	Instância das aulas práticas

Tabela 1 — Conjuntos de instâncias (7 ficheiros, 53 instâncias total).

2. Modelo Matemático

O problema pertence à família **Set Cover** (NP-difícil). A formulação exacta como Programação Inteira Linear (ILP) é:

Variáveis: $x_v \in \{0,1\}$ para cada vértice $v \in V$
Minimizar: $\sum x_v \quad (v \in V)$
Sujeito a: $\sum x_v \geq 1 \quad \forall r \in \Pi'$ (soma sobre os 4 cantos de r)
 $x_v \in \{0,1\}$ (integralidade)

2.1 Relaxação LP como Lower Bound

A **relaxação linear (LP)** — substituir $x_v \in \{0,1\}$ por $0 \leq x_v \leq 1$ — fornece um **lower bound** para o óptimo inteiro. É usada de duas formas: (1) pelo SCIP, que resolve o LP na raiz da árvore de branch-and-bound antes de impor integralidade; (2) no MAC+AC-3, o método **_lower_bound()** implementa um bound fraccionário inspirado no LP: $lb = \text{ceil}(|\text{restrições insatisfeitas}| / \text{cobertura_máx})$, onde cobertura_máx é o número máximo de restrições que qualquer variável livre pode satisfazer simultaneamente.

2.2 Garantia Teórica vs Resultados Práticos

O greedy de cobertura máxima tem garantia de aproximação $H(4) \approx \ln 4 + 1 \approx 2.08$. Na prática, os gaps observados são muito inferiores ao pior caso teórico:

Ficheiro	Gap MaxCov (real)	Garantia $H(4)$	% do máximo teórico
inst_small	+0.40	$\times 2.08$	~19%
inst_1d	+0.60	$\times 2.08$	~29%
inst_grid	+0.00	$\times 2.08$	0%
inst_original	+0.75	$\times 2.08$	~36%
inst_medium	+0.30	$\times 2.08$	~14%
inst_large	+1.40	$\times 2.08$	~67%
inst_adversarial	+0.38	$\times 2.08$	~18%

Tabela 2 — Gap real do Greedy MaxCov vs garantia teórica $H(4)$.

Na prática, o Greedy MaxCov opera muito abaixo do pior caso teórico. Apenas `inst_large` se aproxima de 67% do máximo, esperado para instâncias densas e irregulares.

3. Estratégias Greedy (greedy.py)

Três estratégias greedy foram implementadas, todas suportando cobertura total e parcial via o parâmetro `target_rects`.

3.1 Greedy de Cobertura Máxima

Selecciona iterativamente o vértice que cobre o maior número de retângulos ainda não cobertos. Complexidade: $O(n \cdot |\text{candidates}|)$ por iteração. Gap real: 0.00–1.40 conforme a instância.

```
uncovered <- PI' ; enquanto uncovered != vazio:
    v* <- argmax { |N(v) ∩ uncovered| }
    guards.add(v*) ; uncovered -= N(v*)
```

3.2 Greedy por Grau Estático

Ordena vértices por grau total **uma única vez** no início — $O(|V| \log|V|)$. Não actualiza prioridades. **Inadequado para uso prático**: gap +4.60 em `inst_1d` e +18.60 em `inst_large`.

3.3 Greedy Aleatório com 30 Reinícios

Executa 30 vezes o MaxCov com desempate aleatório (seed configurável); devolve a melhor solução. É o método greedy mais robusto mas **não é sempre ótimo**: gap=+0.20 em `inst_large` e +0.12 em `inst_adversarial`.

3.4 Comparação dos gaps médios.

Estratégia	G.small	G.1d	G.grid	G.orig.	G.med.	G.large	G.adv.	Tempo
MaxCov	+0.40	+0.60	+0.00	+0.75	+0.30	+1.40	+0.38	<1ms
Grau Est.	+0.60	+4.60	+2.17	+1.75	+3.30	+18.60	+5.25	<1ms
Aleat.×30	+0.00	+0.00	+0.00	+0.00	+0.00	+0.20	+0.12	0.6–15.5s

Tabela 3 — Gap médio real (10 runs).

4. Programação Inteira e por Restrições

4.1 ILP com OR-Tools/SCIP (Ortools.py)

Modelo ILP exacto via **pywraplp/SCIP**: para cada $r \in \Pi'$ é criada a restrição $\sum(x[v] \text{ for } v \text{ in } \text{corners}(r)) \geq 1$. Inclui branch-and-bound com relaxação LP na raiz (**ver na Secção 2.1**)

4.2 CP-SAT com OR-Tools (Ortools.py)

Modelo equivalente com **BoolVar** e **AddBoolOr**. O CP-SAT usa propagação de restrições internamente (VSIDS). Os dois solvers concordam sempre no mesmo número de guardas.

4.3 Comparação SCIP vs CP-SAT

Ficheiro	SCIP (ms)	CP-SAT (ms)	Ratio
inst_small	56.7	131.6	×2.3
inst_1d	8.3	15.7	×1.9
inst_grid	8.7	15.1	×1.7
inst_original	9.9	16.4	×1.7
inst_medium	20.1	92.0	×4.6
inst_large	12.9	22.3	×1.7
inst_adversarial	8.1	16.4	×2.0

Tabela 4 — SCIP muito mais rápido que CP-SAT

O SCIP é consistentemente mais rápido que o CP-SAT nas instâncias testadas. O CP-SAT tem overhead de inicialização do motor interno que domina nos problemas pequenos (~131ms em inst_small vs 56.7ms do SCIP). Para inst_medium, o SCIP é ×4.6 mais rápido.

4.4 MAC + AC-3 Manual (mac_ac3.py)

Implementação manual de backtracking com **MAC (Maintaining Arc Consistency)**, sem uso de qualquer módulo externo de programação por restrições. As restrições são n-árias (aridade 4): se todos os cantos menos um de um retângulo estão fixados a 0, o último é forçado a 1 (**forced-one propagation**).

Componente	Função
Classe CSP	Variáveis $x[v] \in \{0,1\}$; restrições n-árias; mapeamento $\text{var} \leftrightarrow \text{restrição}$

ac3()	Fila de restrições; forced-one; retorna False se wipe-out detectado
select_variable_mrv()	MRV (menor domínio) com desempate por grau (mais restrições)
Classe Solver	Branch-and-bound; val=0 primeiro; poda por lower bound LP (Sec. 2.1)
_lower_bound()	ceil(rests.insat. / cob_max) — bound fraccionário inspirado no LP

Tabela 5 — Componentes do solver MAC+AC-3.

Ficheiro	Guardas	Gap	Tempo médio	Notas
inst_small	2.60	+0.00	67ms	Todas as instâncias terminaram
inst_1d	5.00	—	13896ms	Só inst. fáceis; timeout nas difíceis
inst_grid	3.50	+0.00	208ms	Todas as instâncias terminaram
inst_original	3.00	+0.00	58ms	Todas as instâncias terminaram
inst_medium	4.56	+0.00	42ms	Run parcial (1 run disponível)

Tabela 6 — Resultados MAC+AC-3. Em inst_1d apenas instâncias fáceis terminaram.

4.5 SWI-Prolog com CLP(FD) (guards.pl)

Usa **clpfd**. O predicado solve/4 instancia Guards (0/1) e minimiza via labeling([min(Cost), ff, bisect]). A cobertura usa sum(CornerVars, #>=, 1). O solve_partial/5 aceita índices alvo. O solve_file/1 carrega ficheiros .pl gerados pelo Ortools.py.

```
:- use_module(library(clpfd)).
solve(Rects, NV, Guards, Cost) :-
    length(Guards, NV), Guards ins 0..1,
    maplist(coverage_constraint(Guards), Rects),
    sum(Guards, #=, Cost),
    labeling([min(Cost), ff, bisect], Guards).
```

Ficheiro	Guardas	Gap	Tempo médio	Inst. OK	Qualidade
inst_small	2.60	+0.00	~16ms	30/80	Ótimo = CP-SAT
inst_1d	5.00	—	~1387ms	12/40	Timeout selectivo (como MAC)
inst_grid	3.50	+0.00	~98ms	18/48	Ótimo = CP-SAT
inst_original	3.00	+0.00	~15ms	8/24	Ótimo = CP-SAT
inst_medium	4.56	+0.00	~132ms	18/20	Ótimo = CP-SAT

inst_adversarial	4.33	+0.00	~67ms	12/16	Ótimo = CP-SAT
inst_large	N/A	—	N/A	0/5	SWI-Prolog não disponível na máquina de testes

Tabela 7 — Resultados reais do Prolog CLP(FD).

Nota sobre instâncias OK: o Prolog não correu em alguns runs por problema de PATH do executável SWI-Prolog (Prolog_status='SWI-Prolog não instalado' nos CSVs de 1727–1744). Nos runs onde correu, os resultados são sempre ótimos (gap=0.00) e competitivos com o CP-SAT.

5. Programação Dinâmica (dp_guards.py)

Três variantes, seleccionadas automaticamente por `dp_best()` consoante a estrutura da partição.

5.1 DP-1D — Partições Lineares

Detectada por `detect_1d()` quando todos os retângulos partilham a mesma faixa. Aplica recorrência óptima $O(n)$: $dp[j] = \min(dp[j-1]+1, dp[j-2]+1)$. Resultado real: 6.0 guardas, gap=0.00, <0.1ms em `inst_1d` — o único método greedy/DP que resolve `inst_1d` sem timeout.

```
dp[0]=0, dp[1]=1
dp[j] = min(dp[j-1]+1, dp[j-2]+1)
```

5.2 DP-COL e DP-ROW — Profile DP 2D

Para partições 2D, varre fronteiras verticais (COL) ou horizontais (ROW). Estado = bitmask(guardas na fronteira j) $\times$ frozenset(rects não cobertos). Complexidade: $O(K \cdot 2^W \cdot R)$.

5.3 Timeout e Limite Prático de W

Quando o timeout é atingido: a DP devolve a melhor solução parcial encontrada, marcada com `DP_status='timeout'` no CSV. O valor pode ser não-óptimo.

Limite prático de W: com $W \leq 8$ (256 estados), a DP termina em <1ms. Para $W=10-12$ (1024–4096 estados), o tempo cresce para 1–10s. Para $W > 12$, o timeout (5s) activa frequentemente. Em `inst_small`, DP-COL reporta timeout em alguns runs apesar de devolver o resultado correcto — devido a fronteiras com muitos retângulos abertos simultaneamente.

Variante	Quando usada	Gap real	Tempo real	Limite W prático
DP-1D	Estrutura linear (<code>detect_1d</code>)	0.00 em <code>inst_1d</code>	<0.1ms	N/A — $O(n)$
DP-COL	$\max_col \leq \max_row$ (default)	0.40 em <code>inst_small</code>	~0.3ms	$W \leq 10$
DP-ROW	$\max_row < \max_col$	0.00 em <code>inst_medium</code>	~0.4–0.8ms	$W \leq 10$

Tabela 8 — Variantes da DP. DP-COL tem gap+0.40 em `inst_small` (estrutura não linear).

6. Resultados Experimentais (benchmark.py)

Benchmark executado 10 vezes sobre os 7 conjuntos. CP-SAT = referência óptima. Valores são médias dos runs.

6.1 Tabela Completa de Resultados

Ficheiro	G.MaxCov	G.Grau	G.Aleat×30	MAC	DP	SCIP	CP-SAT	Prolog
inst_small	3.00 (+0.40)	3.20 (+0.60)	2.60 (+0.00)	2.60 (+0.00)	3.00 (+0.40)	2.60 (+0.00)	2.60 (+0.00)	2.60 (+0.00)
inst_1d	6.60 (+0.60)	10.60 (+4.60)	6.00 (+0.00)	5.00 (—)	6.00 (+0.00)	6.00 (+0.00)	6.00 (+0.00)	5.00 (—)
inst_grid	3.50 (+0.00)	5.67 (+2.17)	3.50 (+0.00)	3.50 (+0.00)	—	3.50 (+0.00)	3.50 (+0.00)	3.50 (+0.00)
inst_original	3.75 (+0.75)	4.75 (+1.75)	3.00 (+0.00)	3.00 (+0.00)	—	3.00 (+0.00)	3.00 (+0.00)	3.00 (+0.00)
inst_medium	5.20 (+0.30)	8.20 (+3.30)	4.90 (+0.00)	4.56 (+0.00)	—	4.90 (+0.00)	4.90 (+0.00)	4.56 (+0.00)
inst_large	14.20 (+1.40)	31.40 (+18.60)	13.00 (+0.20)	—	—	12.80 (+0.00)	12.80 (+0.00)	N/A
inst_advers.	5.88 (+0.38)	10.75 (+5.25)	5.62 (+0.12)	—	—	5.50 (+0.00)	5.50 (+0.00)	4.33 (+0.00)

Tabela 9 — Guardas médios (gap). MAC e Prolog em inst_1d com timeout selectivo.

6.2 Tempos de Execução — todos os 7 ficheiros

Ficheiro	G.MaxCov	G.Aleat×30	MAC+AC-3	DP	SCIP	CP-SAT	Prolog CLP(FD)
inst_small	<1ms	~603ms	~67ms	~0.4ms	~57ms	~132ms	~16ms
inst_1d	<1ms	~2318ms	~13896ms	<0.1ms	~8ms	~16ms	~1387ms
inst_grid	<1ms	~1303ms	~209ms	—	~9ms	~15ms	~98ms
inst_original	<1ms	~817ms	~58ms	—	~10ms	~16ms	~15ms
inst_medium	<1ms	~2092ms	~42ms	—	~20ms	~92ms	~132ms
inst_large	<1ms	~15522ms	—	—	~13ms	~22ms	N/A
inst_advers.	<1ms	~2507ms	—	—	~8ms	~16ms	~67ms

Tabela 10 — Tempos de execução médios reais (ms).

6.3 Análise

Greedy MaxCov: gap 0.00–1.40, <1ms — equilíbrio velocidade/qualidade.

Greedy Grau: inaceitável em large (+18.60).

Greedy Aleat×30: mais robusto mas não garantidamente óptimo (+0.20 large, +0.12 adversarial).

MAC+AC-3: sempre óptimo quando termina; 13896ms em inst_1d.

DP-1D: óptima e instantânea para lineares; DP-2D pode sofrer timeout.

SCIP: mais rápido que CP-SAT em 7/7 ficheiros.

CP-SAT: overhead de inicialização visível em instâncias pequenas (~132ms).

Prolog CLP(FD): óptimo em todas as instâncias testadas, competitivo em velocidade.

Nenhum método greedy é sempre óptimo. Para optimalidade garantida: SCIP (mais rápido em geral) ou CP-SAT. Para uso prático rápido: Greedy Aleat×30. Para partições lineares: DP-1D.

7. Extensões do Problema (extras.py)

7.1 Extensão 4a — Guardas com Cores

Dois guardas que vigiem o mesmo retângulo devem ter cores distintas. Modela-se como coloração do **grafo de conflito** $G_c = (Guards, E_c)$, onde existe aresta (g_i, g_j) se ambos cobrem pelo menos um retângulo em comum.

Algoritmo	Tipo	Qualidade	Complexidade	Tempo real
DSATUR	Heurístico	$\leq \chi + 1$	$O(Guards ^2)$	<0.1ms (todos os casos)
ILP coloração	Exacto	$\chi(G_c)$ exacto	NP-difícil	0.0–8.3ms (dados reais)

Tabela 11 — Algoritmos de coloração com tempos reais.

Inst.	Rects	Guardas	Arestas conflito	Cores DSATUR	Cores ILP	Tempo ILP
1	16	4	0	1	1	0.5ms
2	25	9	6	3	3	8.3ms
3	36	9	0	1	1	0.0ms
4	24	6	0	1	1	1.0ms
5	28	8	2	2	2	1.5ms

Tabela 12 — Coloração real (5 instâncias de *inst_extensions.txt*).

Resultado chave: o número mínimo de cores é $\chi(G_c)$. DSATUR e ILP concordam sempre. 0 arestas de conflito → 1 cor; 6 arestas → 3 cores. DSATUR <0.1ms, ILP 0–8.3ms.

7.2 Extensão 4b — Guardas com Alcance Alargado D

Um guarda em v cobre todos os retângulos alcançáveis em $\leq D$ arestas no grafo bipartido $V \cup R$. Implementado via **BFS no grafo bipartido** (*coverage_at_distance*). Com $D=0$ ou $D=1$ o comportamento é idêntico ao base.

D	Guardas médios (5 inst.)	Tempo BFS	Redução vs D=0
0	7.2 (base)	0.0–1.0ms	—
1	7.2 (igual)	1.0–1.5ms	0%
2	2.4	1.0–1.5ms	–67%
3	1.0	0.0–2.0ms	–86%
4	1.0	0.0–1.5ms	–86%

Tabela 13 — Impacto real de D (médias de 5 instâncias de *inst_extensions.txt*). BFS <2ms em todos os casos.

O tempo da BFS é sempre <2ms, independentemente de D . Com $D \geq 2$ a redução de guardas é muito significativa (~67–86% nas instâncias testadas).

8. Cobertura Parcial ($\Pi' \subset \Pi$)

Todos os algoritmos recebem o parâmetro **target_rects**. Com cobertura parcial (70% e 80%), todos os greedy melhoram substancialmente. Dados reais do partial_20260521_1955.csv:

Cobertura	G.MaxCov	G.Grau	G.Aleat×30	ILP (ref.)
100% (total)	0.00–1.40	+0.60–18.60	0.00–0.20	ref.
80% (parcial)	+0.14	+1.69	+0.00	ref. (~6.7ms)
70% (parcial)	+0.17	+1.41	+0.00	ref. (~6.3ms)

Tabela 14 — Cobertura parcial (dados reais). Greedy Aleat×30 sempre óptimo na cobertura parcial.

9. Implementação e Arquitectura

Módulo	Responsabilidade	Dependências
greedy.py	3 estratégias greedy + verificação de solução	stdlib
mac_ac3.py	CSP manual + AC-3 + B&B + lower bound LP	OR-Tools
Ortools.py	ILP(SCIP), CP-SAT, exportação .pl para Prolog	OR-Tools
dp_guards.py	DP-1D, DP-COL, DP-ROW + timeout + detect_1d	stdlib
extras.py	Grafo conflito, DSATUR, ILP coloração, BFS alcance D	OR-Tools
guards.pl	CLP(FD): solve/4, solve_partial/5, solve_file/1	SWI-Prolog
benchmark.py	Orquestração, CSV, MAC/Prolog em subprocess	Todos

Tabela 15 — Arquitectura modular do projecto.

O benchmark.py corre MAC+AC-3 em **subprocess separado com timeout 60s**. O Prolog é invocado via subprocess: exporta .pl temporário → chama swipl → captura output.

```
python benchmark.py --all          # todos os ficheiros
python benchmark.py --all --mac    # inclui MAC+AC-3
python benchmark.py --all --partial # cobertura parcial
```

10. Conclusões

Este trabalho avaliou o problema de vigilância de partições retangulares com 8 métodos distintos, testados em 7 conjuntos de instâncias (10 runs, 53 instâncias).

Método	Qualidade (gap)	Velocidade	Escalabilidade
Greedy MaxCov	★★★★☆ (0.00–1.40)	★★★★★	★★★★★
Greedy Grau	★★☆☆☆ (0.00–18.60)	★★★★★	★★★★★
Greedy Aleat.×30	★★★★☆ (0.00–0.20)	★★★★☆	★★★★☆
MAC + AC-3	★★★★★ (0.00 se term.)	★★★☆☆	★★★☆☆
Prog. Dinâmica	★★★★☆ (0.00 lineares)	★★★★★	★★★☆☆
ILP/SCIP	★★★★★ (referência)	★★★★★	★★★★☆
CP-SAT	★★★★★ (referência)	★★★★☆	★★★★☆
Prolog CLP(FD)	★★★★★ (exacto)	★★★☆☆	★★★☆☆

Tabela 16 — Avaliação qualitativa final (dados reais 10 runs).

Destaques e correcções importantes:

- (1) DP em inst_small tem gap +0.40 (não +0.00) — a estrutura não é linear e a DP-COL não encontra o ótimo;
- (2) MAC em inst_1d reporta 5.0 guardas por timeout selectivo, não comparável com CP-SAT=6.0;
- (3) Greedy Aleat×30 não é sempre ótimo (+0.20 large, +0.12 adversarial);
- (4) SCIP é mais rápido que CP-SAT em 7/7 ficheiros;
- (5) Prolog CLP(FD) é ótimo quando SWI-Prolog está disponível; (6) DSATUR <0.1ms, ILP coloração 0–8.3ms, BFS alcance D <2ms em todos os casos.

Recomendação final: SCIP para optimalidade com máxima velocidade; Greedy Aleat×30 para decisões rápidas (gap≈0 em 5/7 ficheiros, <15.5s); DP-1D para partições lineares; Prolog CLP(FD) para instâncias médias com SWI-Prolog disponível; MAC+AC-3 para fins pedagógicos.

Referências

- [1] Chvátal, V. (1979). A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3), 233–235.
- [2] Russell, S. & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*, 4ª ed. Pearson. (Cap. 6: CSP)
- [3] OR-Tools — Google. CP-SAT and SCIP documentation. <https://developers.google.com/optimization>
- [4] Triska, M. The Finite Domain Constraint Solver of SWI-Prolog. *FLOPS 2012*.
- [5] Hochbaum, D. (1997). *Approximation Algorithms for NP-Hard Problems*. PWS Publishing.